

FORMALIZING AND COMPUTING WITH POLYHEDRA IN ROCQ

POLYHEDRA IN LEAN WORKSHOP

Xavier Allamigeon^{1,2}

Joint work w/ Quentin Canu³, Ricardo D. Katz⁴ and Pierre-Yves Strub⁵

¹ Inria ² CMAP, CNRS, École Polytechnique, Institut Polytechnique de Paris

³ Éducation Nationale ⁴ CIFACIS-CONICET ⁵ PQShield

August 26th, 2026

WHY FORMALIZING CONVEX POLYHEDRA?

Pure maths discrete mathematics, combinatorics, algebraic geometry, tropical geometry

Applied maths optimization, operations research, control theory

CS computational geometry, software verification, compilation and program optimization, constraint solving

WHY FORMALIZING CONVEX POLYHEDRA?

Pure maths discrete mathematics, combinatorics, algebraic geometry, tropical geometry

Applied maths optimization, operations research, control theory

CS computational geometry, software verification, compilation and program optimization, constraint solving

Some reasons to formalize convex polyhedra.

- they are extensively used in **critical** applications
 - ⇒ increase the level of trust in polyhedral computations

WHY FORMALIZING CONVEX POLYHEDRA?

Pure maths discrete mathematics, combinatorics, algebraic geometry, tropical geometry

Applied maths optimization, operations research, control theory

CS computational geometry, software verification, compilation and program optimization, constraint solving

Some reasons to formalize convex polyhedra.

- they are extensively used in **critical** applications
 - ⇒ increase the level of trust in polyhedral computations
- they are subject to many **open problems** and **conjectures**
 - ⇒ increase the level of trust in proofs
especially AI generated/assisted proofs

WHY FORMALIZING CONVEX POLYHEDRA?

Pure maths discrete mathematics, combinatorics, algebraic geometry, tropical geometry

Applied maths optimization, operations research, control theory

CS computational geometry, software verification, compilation and program optimization, constraint solving

Some reasons to formalize convex polyhedra.

- they are extensively used in **critical** applications
 - ⇒ increase the level of trust in polyhedral computations
- they are subject to many **open problems** and **conjectures**
 - ⇒ increase the level of trust in proofs
especially AI generated/assisted proofs
- develop **formal experimental mathematics**

WHY FORMALIZING CONVEX POLYHEDRA?

Pure maths discrete mathematics, combinatorics, algebraic geometry, tropical geometry

Applied maths optimization, operations research, control theory

CS computational geometry, software verification, compilation and program optimization, constraint solving

Some reasons to formalize convex polyhedra.

- they are extensively used in **critical** applications
 - ⇒ increase the level of trust in polyhedral computations
- they are subject to many **open problems** and **conjectures**
 - ⇒ increase the level of trust in proofs
especially AI generated/assisted proofs
- develop **formal experimental mathematics**
even more needed with AI

THE HIRSCH CONJECTURE

A simple question. Is there a short path between any two vertices in the graph?

THE HIRSCH CONJECTURE

A simple question. Is there a short path between any two vertices in the graph?

Hirsch conjecture (1957).

The diameter of the vertex-edge graph of a n -dimensional polytope with m facets is bounded by $m - n$.

THE HIRSCH CONJECTURE

A simple question. Is there a short path between any two vertices in the graph?

Hirsch conjecture (1957).

The diameter of the vertex-edge graph of a n -dimensional polytope with m facets is bounded by $m - n$.

Theorem (Santos, 2012). *There is a 43-dimensional polytope with 86 facets and a diameter ≥ 44 .*

Theorem (Matschke, Santos, and Weibel, 2015).

- *There is a 20-dimensional polytope with 40 facets and a diameter ≥ 21 .*
- *There is a 23-dimensional polytope with 46 facets and a diameter ≥ 24 .*



THE HIRSCH CONJECTURE

A simple question. Is there a short path between any two vertices in the graph?

Hirsch conjecture (1957).

The diameter of the vertex-edge graph of a n -dimensional polytope with m facets is bounded by $m - n$.

Theorem (Santos, 2012). *There is a 43-dimensional polytope with 86 facets and a diameter ≥ 44 .*

Theorem (Matschke, Santos, and Weibel, 2015).

- *There is a 20-dimensional polytope with 40 facets and a diameter ≥ 21 .*
- *There is a 23-dimensional polytope with 46 facets and a diameter ≥ 24 .*

Santos' disproof involves computations.

The spindle Q is small enough [...] to be verified computationally, which has been done independently by Edward D. Kim and Julian Pfeifle with the software polymake. Still, we give two computer-free (but not “computation-free”) proofs.



Curry–Howard isomorphism. PROOF $\simeq$ PROGRAM

I.e., checking a proof is **equivalent** to typechecking a program

Curry–Howard isomorphism. PROOF $\simeq$ PROGRAM

I.e., checking a proof is **equivalent** to typechecking a program

- every formula A corresponds to a type A
- a proof of A is a term $x : A$

Example. Modus Ponens

$$\frac{\frac{}{\vdash A \Rightarrow B} \quad \vdash A}{\vdash B} \quad \simeq \quad \frac{f : A \rightarrow B \quad a : A}{f \ a : B}$$

Curry–Howard isomorphism. PROOF $\simeq$ PROGRAM

I.e., checking a proof is **equivalent** to typechecking a program

- every formula A corresponds to a type A
- a proof of A is a term $x : A$

Example. Modus Ponens

$$\frac{\vdash A \Rightarrow B \quad \vdash A}{\vdash B} \quad \simeq \quad \frac{f : A \rightarrow B \quad a : A}{f \ a : B}$$

CIC $\equiv$ Calculus of Inductive Constructions

Intuitionistic logic. In CIC,

- no excluded middle law $P \vee \neg P$
- no double negation elimination $P \Leftrightarrow \neg\neg P$
no reductio ad absurdum
- to show $\exists x. P(x)$, you need to **construct** a x s.t. $P(x)$ holds.

Curry–Howard isomorphism. PROOF $\simeq$ PROGRAM

I.e., checking a proof is **equivalent** to typechecking a program

- every formula A corresponds to a type A
- a proof of A is a term $x : A$

Example. Modus Ponens

$$\frac{\vdash A \Rightarrow B \quad \vdash A}{\vdash B} \quad \simeq \quad \frac{f : A \rightarrow B \quad a : A}{f \ a : B}$$

CIC $\equiv$ Calculus of Inductive Constructions

Intuitionistic logic. In CIC,

- no excluded middle law $P \vee \neg P$
- no double negation elimination $P \Leftrightarrow \neg\neg P$
no reductio ad absurdum
- to show $\exists x. P(x)$, you need to **construct** a x s.t. $P(x)$ holds.

Bonus. CIC has **computing** built in!

“computing” (*) that $x = y$ is a **proof** that x is equal to y .

(*) means reducing terms.



github.com/Coq-Polyhedra/Coq-Polyhedra

- based on the Mathematical Components library (Gonthier, Mahboubi, and Tassi, 2016)
- about 20 kLOC

THIS TALK: THE STORY OF THE COQ-POLYHEDRA PROJECT



github.com/Coq-Polyhedra/Coq-Polyhedra

- based on the Mathematical Components library (Gonthier, Mahboubi, and Tassi, 2016)
- about 20 kLOC

2017: constructive foundations (A., Katz, ITP'17, JAR'19).

How we used the simplex method to formalize basic properties of polyhedra and LP in an effective way.

THIS TALK: THE STORY OF THE COQ-POLYHEDRA PROJECT



github.com/Coq-Polyhedra/Coq-Polyhedra

- based on the Mathematical Components library (Gonthier, Mahboubi, and Tassi, 2016)
- about 20 kLOC

2017: constructive foundations (A., Katz, ITP'17, JAR'19).

How we used the simplex method to formalize basic properties of polyhedra and LP in an effective way.

2020: a first “library” (A., Katz, Strub, IJCAR'20, LMCS'22).

How we formalized key notions on polyhedra and polytopes: basic operations, Minkowski theorem, dimension, face lattice, etc.

THIS TALK: THE STORY OF THE COQ-POLYHEDRA PROJECT



github.com/Coq-Polyhedra/Coq-Polyhedra

- based on the Mathematical Components library (Gonthier, Mahboubi, and Tassi, 2016)
- about 20 kLOC

2017: constructive foundations (A., Katz, ITP'17, JAR'19).

How we used the simplex method to formalize basic properties of polyhedra and LP in an effective way.

2020: a first “library” (A., Katz, Strub, IJCAR'20, LMCS'22).

How we formalized key notions on polyhedra and polytopes: basic operations, Minkowski theorem, dimension, face lattice, etc.

2023: computational proofs (A., Canu, Strub, CPP'23).

How we made Rocq *compute* vertices, graphs, diameters of polytopes and provided a formal verification of Santos' counterexample to the Hirsch conjecture.

The simplex method: fundamental brick of the formalization of convex polyhedra

OVERCOMING THE INTUITIONISTIC NATURE OF ROCQ

Formulas of Rocq live in the sort `Prop`.

	<code>Prop</code>
	True, False
conjunction	$A \wedge B$
disjunction	$A \vee B$
negation	$\sim A$

OVERCOMING THE INTUITIONISTIC NATURE OF ROCQ

Formulas of Rocq live in the sort **Prop**.

	Prop
	True, False
conjunction	$A \wedge B$
disjunction	$A \vee B$
negation	$\sim A$

Let $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$.

Basic question. How to define that the polyhedron $\{x \in \mathbb{R}^n : Ax \geq b\}$ is empty?

OVERCOMING THE INTUITIONISTIC NATURE OF ROCQ

Formulas of Rocq live in the sort `Prop`.

	<code>Prop</code>
	True, False
conjunction	$A \wedge B$
disjunction	$A \vee B$
negation	$\sim A$

Let $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$.

Basic question. How to define that the polyhedron $\{x \in \mathbb{R}^n : Ax \geq b\}$ is empty?

Definition `empty` $A \ b := \sim (\text{exists } x, A * x \geq b) : \text{Prop}.$

OVERCOMING THE INTUITIONISTIC NATURE OF ROCQ

Formulas of Rocq live in the sort `Prop`.

	<code>Prop</code>
	True, False
conjunction	$A \wedge B$
disjunction	$A \vee B$
negation	$\sim A$

Let $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$.

Basic question. How to define that the polyhedron $\{x \in \mathbb{R}^n : Ax \geq b\}$ is empty?

Definition `empty` $A \ b := \sim (\text{exists } x, A *_{\mathbb{m}} x \geq_{\mathbb{m}} b) : \text{Prop}.$

Problems.

- a polyhedron is empty, or non-empty, or what?
- how to extract a point x verifying $A *_{\mathbb{m}} x \geq_{\mathbb{m}} b$ from the negation $\sim (\text{empty } A \ b)$

OVERCOMING THE INTUITIONISTIC NATURE OF ROCQ (2)

Formulas of Rocq live in the sort `Prop`.

+ Boolean algebra, i.e., the type `bool := true | false`

	<code>Prop</code>	<code>bool</code>
	True, False	true, false
conjunction	$A \wedge B$	<code>a && b</code>
disjunction	$A \vee B$	<code>a b</code>
negation	$\sim A$	<code>~~ a</code>

Remark. `bool` behaves like the classical logic!

OVERCOMING THE INTUITIONISTIC NATURE OF ROCQ (2)

Formulas of Rocq live in the sort `Prop`.

+ Boolean algebra, i.e., the type `bool := true | false`

	<code>Prop</code>	<code>bool</code>
	True, False	true, false
conjunction	$A \wedge B$	<code>a && b</code>
disjunction	$A \vee B$	<code>a b</code>
negation	$\sim A$	<code>~~ a</code>

Remark. `bool` behaves like the classical logic!

Boolean reflection methodology (Gonthier et al., 2016).

The relation `reflect P b` means that $P : \text{Prop}$ and $b : \text{bool}$ are equivalent:

- either P holds and $b = \text{true}$,
- or $\sim P$ holds and $b = \text{false}$.

OVERCOMING THE INTUITIONISTIC NATURE OF ROCQ (2)

Formulas of Rocq live in the sort `Prop`.

+ Boolean algebra, i.e., the type `bool := true | false`

	<code>Prop</code>	<code>bool</code>
	True, False	true, false
conjunction	$A \wedge B$	$a \ \&\& \ b$
disjunction	$A \vee B$	$a \ \ b$
negation	$\sim A$	$\sim\sim a$

Remark. `bool` behaves like the classical logic!

Boolean reflection methodology (Gonthier et al., 2016).

The relation `reflect P b` means that $P : \text{Prop}$ and $b : \text{bool}$ are equivalent:

- either P holds and $b = \text{true}$,
- or $\sim P$ holds and $b = \text{false}$.

⇒ if `reflect P b` holds, the Rocq formula P can be manipulated like in classical logic!
case analysis, proof by contradiction, etc

OVERCOMING THE INTUITIONISTIC NATURE OF ROCQ (3)

Formulas of Rocq live in the sort `Prop`.

+ Boolean algebra, i.e., the type `bool := true | false`

	<code>Prop</code>	<code>bool</code>
	True, False	true, false
conjunction	$A \wedge B$	<code>a && b</code>
disjunction	$A \vee B$	<code>a b</code>
negation	$\sim A$	<code>~~ a</code>

Let $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$.

Basic question. How to define that the polyhedron $\{x \in \mathbb{R}^n : Ax \geq b\}$ is empty?

Definition `empty` $A \ b := \sim (\text{exists } x, A * x \geq b) : \text{Prop}.$

OVERCOMING THE INTUITIONISTIC NATURE OF ROCQ (3)

Formulas of Rocq live in the sort `Prop`.

+ Boolean algebra, i.e., the type `bool := true | false`

	<code>Prop</code>	<code>bool</code>
	True, False	true, false
conjunction	$A \wedge B$	<code>a && b</code>
disjunction	$A \vee B$	<code>a b</code>
negation	$\sim A$	<code>~~ a</code>

Let $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$.

Basic question. How to define that the polyhedron $\{x \in \mathbb{R}^n : Ax \geq b\}$ is empty?

Definition `empty` $A \ b := \sim (\text{exists } x, A * x \geq b) : \text{bool}.$

OVERCOMING THE INTUITIONISTIC NATURE OF ROCQ (3)

Formulas of Rocq live in the sort `Prop`.

+ Boolean algebra, i.e., the type `bool := true | false`

	<code>Prop</code>	<code>bool</code>
	True, False	true, false
conjunction	$A \wedge B$	<code>a && b</code>
disjunction	$A \vee B$	<code>a b</code>
negation	$\sim A$	<code>~~ a</code>

Let $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$.

Basic question. How to define that the polyhedron $\{x \in \mathbb{R}^n : Ax \geq b\}$ is empty?

Definition `empty` $A \ b := \sim (\text{exists } x, A * x \geq b) : \text{bool}.$

Remark. Booleans are **computed** by functions written in the language of Rocq!

OVERCOMING THE INTUITIONISTIC NATURE OF ROCQ (3)

Formulas of Rocq live in the sort `Prop`.

+ Boolean algebra, i.e., the type `bool := true | false`

	<code>Prop</code>	<code>bool</code>
	True, False	true, false
conjunction	$A \wedge B$	<code>a && b</code>
disjunction	$A \vee B$	<code>a b</code>
negation	$\sim A$	<code>~~ a</code>

Let $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$.

Basic question. How to define that the polyhedron $\{x \in \mathbb{R}^n : Ax \geq b\}$ is empty?

Definition `empty` $A \ b := \sim (\text{exists } x, A * x \geq b) : \text{bool}.$

Remark. Booleans are **computed** by functions written in the language of Rocq!

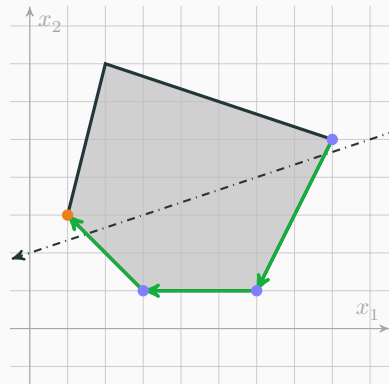
Solution. Implement (Boolean) decision procedures for emptiness, inclusion, boundedness, etc

$\Rightarrow$ one fundamental brick: the simplex method in Rocq

THE SIMPLEX METHOD IN A NUTSHELL

Goal. Minimize $\langle c, x \rangle$ s.t. $Ax \geq b$, $x \in \mathbb{R}^n$,

1. start from a vertex
2. iterate over the **vertex-edge** graph while decreasing the objective function
3. up to finding an **optimal** vertex

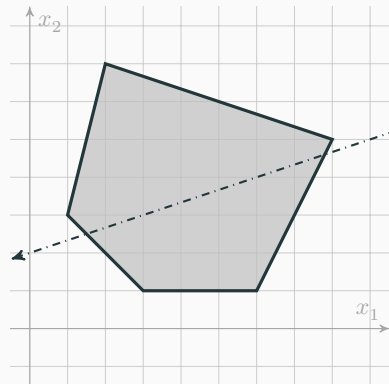


THE SIMPLEX METHOD IN A NUTSHELL

Goal. Minimize $\langle c, x \rangle$ s.t. $Ax \geq b$, $x \in \mathbb{R}^n$,

1. start from a vertex
2. iterate over the **vertex-edge** graph while decreasing the objective function
3. up to finding an **optimal** vertex

Three ingredients.



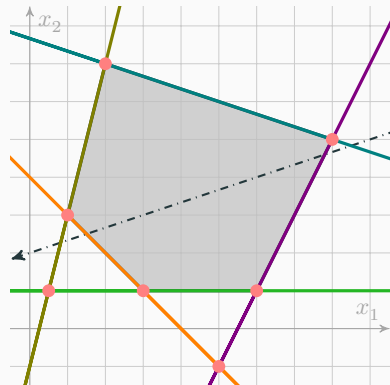
THE SIMPLEX METHOD IN A NUTSHELL

Goal. Minimize $\langle c, x \rangle$ s.t. $Ax \geq b$, $x \in \mathbb{R}^n$,

1. start from a vertex
2. iterate over the **vertex-edge** graph while decreasing the objective function
3. up to finding an **optimal** vertex

Three ingredients.

- bases encode **vertices**



Definition. A basis is a set I of n supporting hyperplanes $A_i x = b_i$ whose intersection is a **single point** x^I .

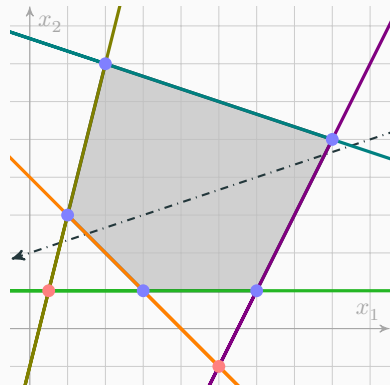
THE SIMPLEX METHOD IN A NUTSHELL

Goal. Minimize $\langle c, x \rangle$ s.t. $Ax \geq b$, $x \in \mathbb{R}^n$,

1. start from a vertex
2. iterate over the **vertex-edge** graph while decreasing the objective function
3. up to finding an **optimal** vertex

Three ingredients.

- bases encode **vertices**



Definition. A basis is a set I of n supporting hyperplanes $A_i x = b_i$ whose intersection is a **single point** x^I . The basis is feasible if x^I belongs to the polyhedron. In this case, x^I is a **vertex**.

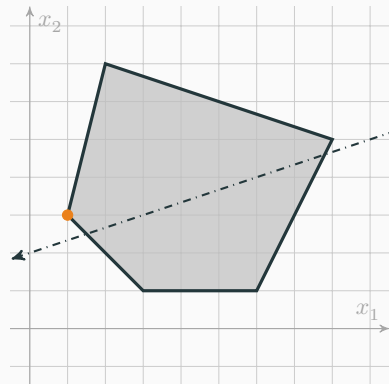
THE SIMPLEX METHOD IN A NUTSHELL

Goal. Minimize $\langle c, x \rangle$ s.t. $Ax \geq b$, $x \in \mathbb{R}^n$,

1. start from a vertex
2. iterate over the **vertex-edge** graph while decreasing the objective function
3. up to finding an **optimal** vertex

Three ingredients.

- bases encode **vertices**
- reduced costs determine **optimality**



Reduced cost vector. The reduced cost vector at basis I is defined as $u^I := A_I^{-T} c_I$ on I , 0 out of I .

When non-negative, it provides a feasible (optimal) element of the dual LP:

$$\text{maximize } \langle b, u \rangle \quad \text{subject to} \quad A^T u = c, \quad u \geq 0, \quad u \in \mathbb{R}^m.$$

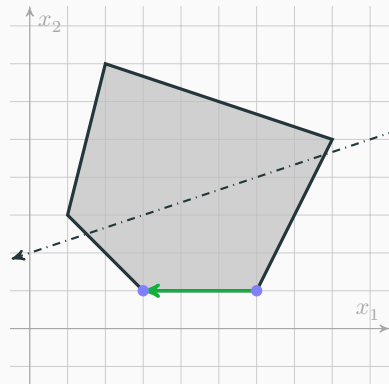
THE SIMPLEX METHOD IN A NUTSHELL

Goal. Minimize $\langle c, x \rangle$ s.t. $Ax \geq b$, $x \in \mathbb{R}^n$,

1. start from a vertex
2. iterate over the **vertex-edge** graph while decreasing the objective function
3. up to finding an **optimal** vertex

Three ingredients.

- bases encode **vertices**
- reduced costs determine **optimality**
- pivoting switches from a **vertex** to **another**



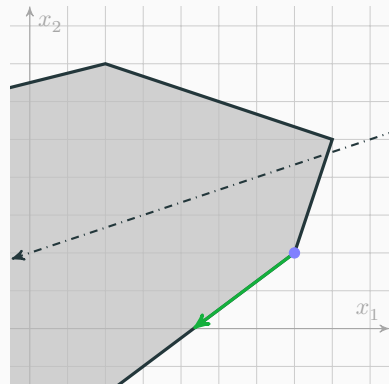
THE SIMPLEX METHOD IN A NUTSHELL

Goal. Minimize $\langle c, x \rangle$ s.t. $Ax \geq b$, $x \in \mathbb{R}^n$,

1. start from a vertex
2. iterate over the **vertex-edge** graph while decreasing the objective function
3. up to finding an **optimal** vertex

Three ingredients.

- bases encode **vertices**
- reduced costs determine **optimality**
- pivoting switches from a **vertex** to **another** or finds a **direction** along which the LP is unbounded



Unboundedness. A direction vector $d \in \mathbb{R}^n$ satisfying

$$Ad \geq 0, \quad \langle c, d \rangle < 0$$

This shows that the LP is unbounded: $A(x + td) \geq b$ and $\langle c, x + td \rangle \rightarrow -\infty$ as $t \rightarrow \infty$.

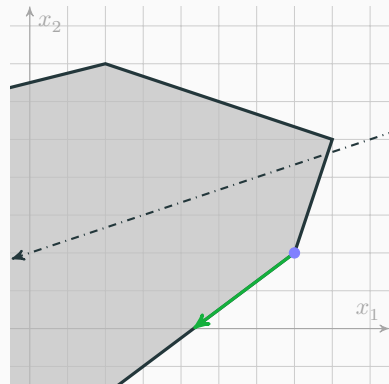
THE SIMPLEX METHOD IN A NUTSHELL

Goal. Minimize $\langle c, x \rangle$ s.t. $Ax \geq b$, $x \in \mathbb{R}^n$,

1. start from a vertex
2. iterate over the **vertex-edge** graph while decreasing the objective function
3. up to finding an **optimal** vertex

Three ingredients.

- bases encode **vertices**
- reduced costs determine **optimality**
- pivoting switches from a **vertex** to **another** or finds a **direction** along which the LP is unbounded



Summary. The (phase II) simplex method starts from a feasible basis, and:

- either finds an **optimal** feasible basis I , with nonnegative reduced cost u^I ;
- or a **direction** d along which the LP is unbounded.

Main idea. Apply the simplex method to the LP

$$\text{maximize } \langle b, u \rangle \quad \text{subject to } A^T u = 0, u \geq 0, u \in \mathbb{R}^m$$

whose dual is simply

$$\text{minimize } 0 \quad \text{subject to } Ax \geq b, x \in \mathbb{R}^n.$$

Main idea. Apply the simplex method to the LP

$$\text{maximize } \langle b, u \rangle \quad \text{subject to } A^T u = 0, u \geq 0, u \in \mathbb{R}^m$$

whose dual is simply

$$\text{minimize } 0 \quad \text{subject to } Ax \geq b, x \in \mathbb{R}^n.$$

Suppose that we have formalized the simplex method in Rocq:

```
Definition empty A b :=  
  match simplex [A^T -A^T 1:%M] 0 (-b) I_0 with  
  | Unbounded d => true  
  | Optimal_basis I => false  
end.
```


EMPTINESS AS A BOOLEAN PREDICATE

Main idea. Apply the simplex method to the LP

$$\text{maximize } \langle b, u \rangle \quad \text{subject to } A^T u = 0, u \geq 0, u \in \mathbb{R}^m$$

whose dual is simply

$$\text{minimize } 0 \quad \text{subject to } Ax \geq b, x \in \mathbb{R}^n.$$

Suppose that we have formalized the simplex method in Rocq:

```
Definition empty A b :=  
  match simplex [A^T -A^T 1:%M] 0 (-b) I_0 with  
  | Unbounded d => true  
  | Optimal_basis I => false  
end.
```

We get...

```
Fact: empty A b = true  
-> exists d, [/ \ d >= 0, A^T *m d = 0 & '[b,d] > 0].
```

```
Fact: empty A b = false -> exists x, A *m x >= b.
```

EMPTINESS AS A BOOLEAN PREDICATE

Main idea. Apply the simplex method to the LP

maximize $\langle b, u \rangle$ subject to $A^T u = 0, u \geq 0, u \in \mathbb{R}^m$

whose dual is simply

minimize 0 subject to $Ax \geq b, x \in \mathbb{R}^n$.

Suppose that we have formalized the simplex method in Rocq:

```
Definition empty A b :=  
  match simplex [A^T -A^T 1:%M] 0 (-b) I_0 with  
  | Unbounded d => true  
  | Optimal_basis I => false  
end.
```

We get...

```
Fact: empty A b = true  
-> exists d, [/ \ d >= 0, A^T *m d = 0 & '[b,d] > 0].
```

```
Fact: empty A b = false -> exists x, A *m x >= b.
```

EMPTINESS AS A BOOLEAN PREDICATE

Main idea. Apply the simplex method to the LP

$$\text{maximize } \langle b, u \rangle \quad \text{subject to } A^T u = 0, u \geq 0, u \in \mathbb{R}^m$$

whose dual is simply

$$\text{minimize } 0 \quad \text{subject to } Ax \geq b, x \in \mathbb{R}^n.$$

Suppose that we have formalized the simplex method in Rocq:

```
Definition empty A b :=  
  match simplex [A^T -A^T 1:%M] 0 (-b) I_0 with  
  | Unbounded d => true  
  | Optimal_basis I => false  
end.
```

We get...

```
Fact: empty A b = true  
  -> exists d, [/ \ d >=m 0, A^T *m d = 0 & '[b,d] > 0].
```

```
Fact: empty A b = false -> exists x, A *m x >=m b.
```

we build x from
the reduced costs of basis I

Main idea. Apply the simplex method to the LP

$$\text{maximize } \langle b, u \rangle \quad \text{subject to } A^T u = 0, u \geq 0, u \in \mathbb{R}^m$$

whose dual is simply

$$\text{minimize } 0 \quad \text{subject to } Ax \geq b, x \in \mathbb{R}^n.$$

Suppose that we have formalized the simplex method in RocQ:

```
Definition empty A b :=  
  match simplex [A^T -A^T 1:%M] 0 (-b) I_0 with  
  | Unbounded d => true  
  | Optimal_basis I => false  
end.
```

We get... Farkas Lemma for free!

```
Fact: empty A b = true  
  -> exists d, [/ \ d >= 0, A^T *m d = 0 & '[b,d] > 0].
```

```
Fact: empty A b = false -> exists x, A *m x >= b.
```

Farkas Lemma.

- There is $x \in \mathbb{R}^n$ such that $Ax \geq b$,
- or there is $d \in \mathbb{R}_{\geq 0}^m$ such that $A^T d = 0$ and $\langle b, d \rangle > 0$.

but not both.

Main idea. Apply the simplex method to the LP

$$\text{maximize } \langle b, u \rangle \quad \text{subject to} \quad A^T u = 0, \quad u \geq 0, \quad u \in \mathbb{R}^m$$

whose dual is simply

$$\text{minimize } 0 \quad \text{subject to} \quad Ax \geq b, \quad x \in \mathbb{R}^n.$$

Suppose that we have formalized the simplex method in Rocq:

```
Definition empty A b :=
  match simplex [A^T -A^T 1:%M] 0 (-b) I_0 with
  | Unbounded d => true
  | Optimal_basis I => false
  end.
```

We get... Farkas Lemma for free!

Fact: `empty A b = true`
 $\rightarrow \text{exists } d, [\wedge d \geq 0, A^T * d = 0 \ \& \ '[b,d] > 0].$

Fact: `empty A b = false` $\rightarrow \text{exists } x, A * x \geq b.$

Farkas Lemma.

- There is $x \in \mathbb{R}^n$ such that $Ax \geq b$,
- or there is $d \in \mathbb{R}_{\geq 0}^m$ such that $A^T d = 0$ and $\langle b, d \rangle > 0$.

but not both.

Remark. d is an infeasibility certificate:
 if $x \in \mathbb{R}^n$ satisfies $Ax \geq b$,

$$0 = \langle A^T d, x \rangle = \langle d, Ax \rangle \geq \langle d, b \rangle > 0,$$

i.e., the “not both”-part is trivial.

EMPTINESS AS A BOOLEAN PREDICATE

Main idea. Apply the simplex method to the LP

$$\text{maximize } \langle b, u \rangle \quad \text{subject to } A^T u = 0, u \geq 0, u \in \mathbb{R}^m$$

whose dual is simply

$$\text{minimize } 0 \quad \text{subject to } Ax \geq b, x \in \mathbb{R}^n.$$

Suppose that we have formalized the simplex method in Rocq:

```
Definition empty A b :=  
  match simplex [A^T -A^T 1:%M] 0 (-b) I_0 with  
  | Unbounded d => true  
  | Optimal_basis I => false  
end.
```

Idiomatic formulation. Using Boolean reflection:

Theorem emptyPn A b :
 reflect (exists x, A * x >= b) (~~ empty A b).

Theorem emptyP A b :
 reflect (exists d, [/\ d >= 0, A^T * d = 0 & '[b,d] > 0]) (empty A b).

Farkas Lemma.

- There is $x \in \mathbb{R}^n$ such that $Ax \geq b$,
- or there is $d \in \mathbb{R}_{\geq 0}^m$ such that $A^T d = 0$ and $\langle b, d \rangle > 0$.

but not both.

Main idea. Apply the simplex method to the LP

$$\text{maximize } \langle b, u \rangle \quad \text{subject to} \quad A^T u = 0, \quad u \geq 0, \quad u \in \mathbb{R}^m$$

whose dual is simply

$$\text{minimize } 0 \quad \text{subject to} \quad Ax \geq b, \quad x \in \mathbb{R}^n.$$

Suppose that we have formalized the simplex method in Rocq:

Definition `empty A b :=`

```
match simplex A b with
| Unbounded => true
| Optimal _ => false
end.
```

We do not execute the simplex method.

We only use the facts that:

- it is typechecked (in particular, it terminates)
- it is proved to be correct

We get... Farkas Lemma

Fact: `empty A b = true`

`-> exists d, [/ \ d >= 0, A^T * m d = 0 & ' [b,d] > 0].`

Fact: `empty A b = false`

`-> exists x, A * m x >= m b.`

Farkas Lemma.

- There is $x \in \mathbb{R}^n$ such that $Ax \geq b$,
- or there is $d \in \mathbb{R}^m$ such that $A^T d = 0$ and $\langle b, d \rangle > 0$.

but not both.

Dantzig's lexicographic rule (Dantzig et al., 1955).

We consider a perturbation of the LP involving $0 < \varepsilon \ll 1$:

$$\text{minimize } \langle c, x \rangle \quad \text{subject to } Ax \geq \tilde{b}, \quad \text{where } \tilde{b}_i := b_i - \varepsilon^i$$

Dantzig's lexicographic rule (Dantzig et al., 1955).

We consider a perturbation of the LP involving $0 < \varepsilon \ll 1$:

$$\text{minimize } \langle c, x \rangle \quad \text{subject to } Ax \geq \tilde{b}, \quad \text{where } \tilde{b}_i := b_i - \varepsilon^i$$

$\Rightarrow$ lex-feasible bases $:=$ the feasible bases of $\{x \in \mathbb{R}^n : Ax \geq \tilde{b}\}$

+ basic points $\equiv$ perturbations of the basic points of $\{x \in \mathbb{R}^n : Ax \geq b\}$

Dantzig's lexicographic rule (Dantzig et al., 1955).

We consider a perturbation of the LP involving $0 < \varepsilon \ll 1$:

$$\text{minimize } \langle c, x \rangle \quad \text{subject to} \quad Ax \geq \tilde{b}, \quad \text{where } \tilde{b}_i := b_i - \varepsilon^i$$

$\Rightarrow$ lex-feasible bases $:=$ the feasible bases of $\{x \in \mathbb{R}^n : Ax \geq \tilde{b}\}$

+ basic points $\equiv$ perturbations of the basic points of $\{x \in \mathbb{R}^n : Ax \geq b\}$

Lemma `no_degenerate_basis` (I I' : lex_feasible_basis) :
point_of_basis I = point_of_basis I' $\rightarrow$ I = I'.

FORMALIZATION OF THE SIMPLEX METHOD: DEALING WITH TERMINATION

Dantzig's lexicographic rule (Dantzig et al., 1955).

We consider a perturbation of the LP involving $0 < \varepsilon \ll 1$:

$$\text{minimize } \langle c, x \rangle \quad \text{subject to} \quad Ax \geq \tilde{b}, \quad \text{where } \tilde{b}_i := b_i - \varepsilon^i$$

$\Rightarrow$ lex-feasible bases $\equiv$ the feasible bases of $\{x \in \mathbb{R}^n : Ax \geq \tilde{b}\}$

+ basic points $\equiv$ perturbations of the basic points of $\{x \in \mathbb{R}^n : Ax \geq b\}$

Lemma `no_degenerate_basis` (`I I' : lex_feasible_basis`) :
`point_of_basis I = point_of_basis I' -> I = I'.`

Easy implementation. The perturbation of b into $\tilde{b}$ is done symbolically

- every perturbed real $\tilde{v}$ is a polynomial in ε of degree $\leq m$

$$v + v_1\varepsilon + v_2\varepsilon^2 + \dots + v_m\varepsilon^m$$

encoded as a row vector $(v, v_1, \dots, v_m)$.

- the usual order $\leq$ is replaced by the lexicographic order $\leq_{\text{lex}}$

$\Rightarrow$ minor change in the formalization of the pivoting operation

Formalized library of polyhedra

We implement the type 'hpoly_n of H-polyhedra, i.e., polyhedra given by:

```
Record hpoly := HPoly { m : nat; A : 'M_(m,n); b : 'cV_m }.
```

representing the set of x satisfying $A * m \ x \geq m \ b$.

We implement the type 'hpoly_n of H-polyhedra, i.e., polyhedra given by:

```
Record hpoly := HPoly { m : nat; A : 'M_(m,n); b : 'cV_m }.
```

representing the set of x satisfying $A \cdot_m x \geq_m b$.

We rely on the simplex method to define a minimal set of primitives over 'hpoly_n.

Example. Given two H-polyhedra $P, Q : \text{'poly_n}$, the inclusion

```
Definition poly_subset P Q := ... : bool
```

is defined by checking $\min_{x \in P} \langle \alpha, x \rangle \geq \beta$ for every defining inequality $\langle \alpha, x \rangle \geq \beta$ of Q .

We implement the type 'hpoly_n of H-polyhedra, i.e., polyhedra given by:

Record hpoly := HPoly { m : nat; A : 'M_(m,n); b : 'cV_m }.

representing the set of x satisfying $A \cdot x \geq b$.

We rely on the simplex method to define a minimal set of primitives over 'hpoly_n.

Example. Given two H-polyhedra P, Q : 'poly_n, the inclusion

Definition poly_subset P Q := ... : bool

is defined by checking $\min_{x \in P} \langle \alpha, x \rangle \geq \beta$ for every defining inequality $\langle \alpha, x \rangle \geq \beta$ of Q.

Corollary of the correctness of the simplex method.

Lemma poly_subsetP (P Q : 'hpoly_n) :
reflect (forall x, x \in P -> x \in Q) (poly_subset P Q).

Lemma poly_subsetPn (P Q : 'hpoly_n) :
reflect (exists2 x, x \in P & x \notin Q) (~~ (poly_subset P Q)).

THE TYPE OF POLYHEDRA

We build the type 'poly_n of polyhedra as a quotient of the following relation:

```
Definition poly_equiv (P Q : 'hpoly_n) :=  
  (poly_subset P Q) && (poly_subset Q P).
```

that states that P and Q represent the same set of points:

```
Lemma poly_equivP {P Q : 'hpoly[R]_n} :  
  reflect (forall x, x \in P <-> x \in Q) (poly_equiv P Q).
```


THE TYPE OF POLYHEDRA

We build the type 'poly_n of polyhedra as a quotient of the following relation:

```
Definition poly_equiv (P Q : 'hpoly_n) :=  
  (poly_subset P Q) && (poly_subset Q P).
```

that states that P and Q represent the same set of points:

```
Lemma poly_equivP {P Q : 'hpoly[R]_n} :  
  reflect (forall x, x \in P <-> x \in Q) (poly_equiv P Q).
```

Building the quotient type (Cohen, 2013).

- define a canonical representative canon P : 'hpoly_n for each class:

```
Lemma canon_eq (P Q : 'hpoly_n) :  
  poly_equiv P Q <-> canon P = canon Q.
```

- every “class” consists of the representative of the class

```
Inductive poly := Poly { P : 'hpoly_n;  
  _ : P = canon P }.
```

THE TYPE OF POLYHEDRA

We build the type 'poly_n of polyhedra as a quotient of the following relation:

Definition `poly_equiv` (P Q : 'hpoly_n) :=
 (poly_subset P Q) && (poly_subset Q P).

that states that P and Q represent the same set of points:

Lemma `poly_equivP` {P Q : 'hpoly[R]_n} :
 reflect (forall x, x \in P <-> x \in Q) (poly_equiv P Q).

Building the quotient type (Cohen, 2013).

- define a canonical representative `canon P : 'hpoly_n` for each class:

Lemma `canon_eq` (P Q : 'hpoly_n) :
 poly_equiv P Q <-> canon P = canon Q.

- every “class” consists of the representative of the class

Inductive `poly` := Poly { P : 'hpoly_n;
 _ : P = canon P }.

Remark. The representative has no special mathematical meaning or structure.

The type `'poly_n` of polyhedra.

- lift the membership predicate from `'hpoly_n` to `'poly_n`:
 $x \setminus \text{in } P \equiv x \text{ belongs to the canonical representative}$
- equality between polyhedra is equivalent to extensional equality of their membership predicates

Lemma `poly_eqP` ($P\ Q : \text{'poly_n}$) :
 $P = Q \leftrightarrow (\text{forall } x, x \setminus \text{in } P \leftrightarrow x \setminus \text{in } Q).$

The type 'poly_n of polyhedra.

- lift the membership predicate from 'hpoly_n to 'poly_n:
 $x \setminus \text{in } P \equiv x \text{ belongs to the canonical representative}$
- equality between polyhedra is equivalent to extensional equality of their membership predicates

Lemma poly_eqP (P Q : 'poly_n) :
P = Q $\leftrightarrow$ (forall x, x \setminus \text{in } P $\leftrightarrow$ x \setminus \text{in } Q).

Features. Full set of operations/constructions over the type 'poly_n

- subset relation P ' $\leq$ ' Q, intersection P '&' Q of polyhedra, ...
- halfspaces [hs e], hyperplanes [hp e], points [pt x], line segments [segm x & y], etc
- standard constructions like convex hull conv V, projection, image under linear map, etc
- basic theorems like Farkas Lemma, Strong Duality Theorem, separation by hyperplanes

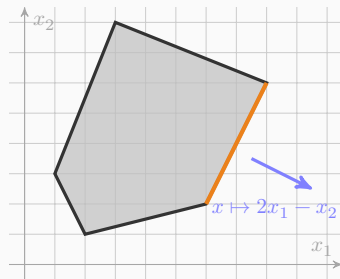
FORMALIZING FACES

Let $\mathcal{P} \subset \mathbb{R}^n$ be a polyhedron.

Definition. A set $\mathcal{F}$ is a **face** of $\mathcal{P}$ if

$$\mathcal{F} = \emptyset$$

or $\mathcal{F}$ = the set of points $x \in \mathcal{P}$ minimizing
the function $x \mapsto \langle c, x \rangle$
(for some $c \in \mathbb{R}^n$)



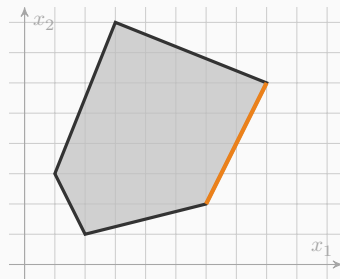
FORMALIZING FACES

Let $\mathcal{P} \subset \mathbb{R}^n$ be a polyhedron.

Definition. A set $\mathcal{F}$ is a **face** of $\mathcal{P}$ if

$$\mathcal{F} = \emptyset$$

or $\mathcal{F}$ = the set of points $x \in \mathcal{P}$ minimizing
the function $x \mapsto \langle c, x \rangle$
(for some $c \in \mathbb{R}^n$)



Suppose that $\mathcal{P}$ is given by the H-representation $Ax \geq b$, where $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$.

Theorem. A set $\mathcal{F}$ is face of $\mathcal{P}$ if, and only if,

$$\mathcal{F} = \emptyset$$

or $\mathcal{F} = \mathcal{P} \cap \{x \in \mathbb{R}^n : A_i x = b_i \text{ for all } i \in I\}$ for some $I \subset \{1, \dots, m\}$.

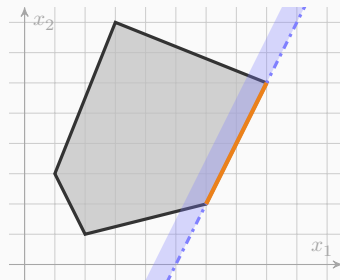
FORMALIZING FACES

Let $\mathcal{P} \subset \mathbb{R}^n$ be a polyhedron.

Definition. A set $\mathcal{F}$ is a **face** of $\mathcal{P}$ if

$$\mathcal{F} = \emptyset$$

or $\mathcal{F}$ = the set of points $x \in \mathcal{P}$ minimizing
the function $x \mapsto \langle c, x \rangle$
(for some $c \in \mathbb{R}^n$)



Suppose that $\mathcal{P}$ is given by the H-representation $Ax \geq b$, where $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$.

Theorem. A set $\mathcal{F}$ is face of $\mathcal{P}$ if, and only if,

$$\mathcal{F} = \emptyset$$

or $\mathcal{F} = \mathcal{P} \cap \{x \in \mathbb{R}^n : A_i x = b_i \text{ for all } i \in I\}$ for some $I \subset \{1, \dots, m\}$.

active inequalities

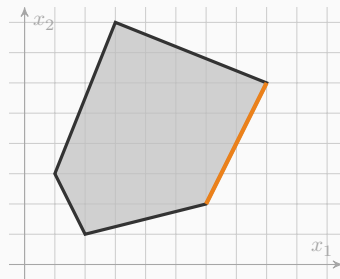
FORMALIZING FACES

Let $\mathcal{P} \subset \mathbb{R}^n$ be a polyhedron.

Definition. A set $\mathcal{F}$ is a **face** of $\mathcal{P}$ if

$$\mathcal{F} = \emptyset$$

or $\mathcal{F}$ = the set of points $x \in \mathcal{P}$ minimizing
the function $x \mapsto \langle c, x \rangle$
(for some $c \in \mathbb{R}^n$)



Suppose that $\mathcal{P}$ is given by the H-representation $Ax \geq b$, where $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$.

Theorem. A set $\mathcal{F}$ is face of $\mathcal{P}$ if, and only if,

$$\mathcal{F} = \emptyset$$

or $\mathcal{F} = \mathcal{P} \cap \{x \in \mathbb{R}^n : A_i x = b_i \text{ for all } i \in I\}$ for some $I \subset \{1, \dots, m\}$.

Remark. The characterization above involves a certain **H-representation** of $\mathcal{P}$,
while the notion of face is purely **geometric**.

Proposition. *If $\mathcal{F}$ is a face of $\mathcal{P}$, then any face of $\mathcal{F}$ is a face of $\mathcal{P}$.*

Proposition. *If $\mathcal{F}$ is a face of $\mathcal{P}$, then any face of $\mathcal{F}$ is a face of $\mathcal{P}$.*

Proof. Assume $\mathcal{P}$ is represented by the inequality system $Ax \geq b$, and take I such that

$$\mathcal{F} = \mathcal{P} \cap \{x \in \mathbb{R}^n : A_i x = b_i \text{ for all } i \in I\}$$

Proposition. *If $\mathcal{F}$ is a face of $\mathcal{P}$, then any face of $\mathcal{F}$ is a face of $\mathcal{P}$.*

Proof. Assume $\mathcal{P}$ is represented by the inequality system $Ax \geq b$, and take I such that

$$\mathcal{F} = \mathcal{P} \cap \{x \in \mathbb{R}^n : A_i x = b_i \text{ for all } i \in I\}$$

Let $\mathcal{F}'$ be a nonempty face of $\mathcal{F}$. We apply the previous characterization of faces on the following H-representation of $\mathcal{F}$:

$$\begin{aligned} Ax &\geq b \\ -A_i x &\geq -b_i \quad \text{for } i \in I. \end{aligned}$$

Proposition. *If $\mathcal{F}$ is a face of $\mathcal{P}$, then any face of $\mathcal{F}$ is a face of $\mathcal{P}$.*

Proof. Assume $\mathcal{P}$ is represented by the inequality system $Ax \geq b$, and take I such that

$$\mathcal{F} = \mathcal{P} \cap \{x \in \mathbb{R}^n : A_i x = b_i \text{ for all } i \in I\}$$

Let $\mathcal{F}'$ be a nonempty face of $\mathcal{F}$. We apply the previous characterization of faces on the following H-representation of $\mathcal{F}$:

$$\begin{aligned} Ax &\geq b \\ -A_i x &\geq -b_i \quad \text{for } i \in I. \end{aligned}$$

We get

$$\begin{aligned} \mathcal{F}' &= \mathcal{F} \cap \{x \in \mathbb{R}^n : A_i x = b_i \text{ for all } i \in I^+, -A_i x = -b_i \text{ for all } i \in I^-\} \quad \text{for some } I^+ \subset [m], I^- \subset I \\ &= \mathcal{P} \cap \{x \in \mathbb{R}^n : A_i x = b_i \text{ for all } i \in I \cup I^+ \cup I^-\} \end{aligned}$$

and conclude that $\mathcal{F}'$ is a face of $\mathcal{P}$.

Proposition. *If $\mathcal{F}$ is a face of $\mathcal{P}$, then any face of $\mathcal{F}$ is a face of $\mathcal{P}$.*

Proof. Assume $\mathcal{P}$ is represented by the inequality system $Ax \geq b$, and take I such that

$$\mathcal{F} = \mathcal{P} \cap \{x \in \mathbb{R}^n : A_i x = b_i \text{ for all } i \in I\}$$

Let $\mathcal{F}'$ be a nonempty face of $\mathcal{F}$. We apply the previous characterization of faces on the following H-representation of $\mathcal{F}$:

$$\begin{aligned} Ax &\geq b \\ -A_i x &\geq -b_i \quad \text{for } i \in I. \end{aligned}$$

We get

$$\begin{aligned} \mathcal{F}' &= \mathcal{F} \cap \{x \in \mathbb{R}^n : A_i x = b_i \text{ for all } i \in I^+, -A_i x = -b_i \text{ for all } i \in I^-\} \quad \text{for some } I^+ \subset [m], I^- \subset I \\ &= \mathcal{P} \cap \{x \in \mathbb{R}^n : A_i x = b_i \text{ for all } i \in I \cup I^+ \cup I^-\} \end{aligned}$$

and conclude that $\mathcal{F}'$ is a face of $\mathcal{P}$.

arbitrary H-representation

vs

specific H-representation

Idea. When dealing with the faces of a polyhedron, and possibly with the faces of these faces, etc,

1. first set a common **ambient H-representation**, i.e., a finite set of inequalities **base**.
2. then manipulate faces within this H-representation, possibly with more active inequalities.

Idea. When dealing with the faces of a polyhedron, and possibly with the faces of these faces, etc,

1. first set a common **ambient H-representation**, i.e., a finite set of inequalities **base**.
2. then manipulate faces within this H-representation, possibly with more active inequalities.

We introduce the following dependent type

$\{\text{poly base}\} \equiv$ subtype of 'poly_n of the polyhedra which can be over the H-representation **base**

Inductive **poly_base** base :=
 PolyBase { pval :> 'poly_n ;
 _ : has_base base pval }.

Definition **has_base** base Q :=
 Q != [poly0] -> **exists** I : {fsubset base}, Q = 'P^=(base; I).

Idea. When dealing with the faces of a polyhedron, and possibly with the faces of these faces, etc,

1. first set a common **ambient H-representation**, i.e., a finite set of inequalities **base**.
2. then manipulate faces within this H-representation, possibly with more active inequalities.

We introduce the following dependent type

$\{\text{poly } \text{base}\} \equiv$ subtype of 'poly_n of the polyhedra which can be over the H-representation **base**

Inductive **poly_base** base :=
 PolyBase { pval :> 'poly_n ;
 _ : has_base base pval }.

Definition **has_base** base Q :=
 Q != [poly0] -> **exists** I : {fsubset base}, Q = 'P^=(base; I).

$\equiv$ polyhedron defined by the inequalities in base
+ the equalities in the subset $I \subset \text{base}$

Idea. When dealing with the faces of a polyhedron, and possibly with the faces of these faces, etc,

1. first set a common **ambient H-representation**, i.e., a finite set of inequalities **base**.
2. then manipulate faces within this H-representation, possibly with more active inequalities.

We introduce the following dependent type

$\{\text{poly base}\} \equiv$ subtype of $'\text{poly_n}$ of the polyhedra which can be over the H-representation base

Inductive **poly_base** base :=
 PolyBase { pval $\Rightarrow$ $'\text{poly_n}$;
 _ : has_base base pval }.

Definition **has_base** base Q :=
 Q != [poly0] $\rightarrow$ **exists** I : {fsubset base}, Q = $'P^=(\text{base}; I)$.

$\equiv$ polyhedron defined by the inequalities in base
 + the equalities in the subset $I \subset \text{base}$

Notation $''\{\text{poly } ' \text{ base } '\}'' := \text{poly_base base}.$

Implicit coercion. Given $P : \{\text{poly base}\}$, the underlying polyhedron is given by $P : '\text{poly_n}$.

A DEFINITION OF FACES

Over the type `{poly base}`, the definition of the faces is **straightforward!**

Definition `poly_base_face_set` ($P : \{\text{poly base}\}$) :=
[set $F : \{\text{poly base}\} \mid F \leq P$].

A DEFINITION OF FACES

Over the type `{poly base}`, the definition of the faces is **straightforward**!

Definition `poly_base_face_set` (`P : {poly base}`) :=
[`set F : {poly base} | F '<=' P`].

This definition is **independent** of the choice of base:

Lemma `face_set_morph` `base1 base2 (P1 : {poly base1}) (P2 : {poly base2}) :`
`P1 = P2 => 'poly_n -> poly_base_face_set P1 = poly_base_face_set P2 => {fset 'poly_n}.`

A DEFINITION OF FACES

Over the type `{poly base}`, the definition of the faces is **straightforward**!

Definition `poly_base_face_set` (`P : {poly base}`) :=
[set `F : {poly base}` | `F '≤' P`].

This definition is **independent** of the choice of base:

Lemma `face_set_morph` `base1 base2` (`P1 : {poly base1}`) (`P2 : {poly base2}`) :
`P1 = P2` `:> 'poly_n` `->` `poly_base_face_set P1 = poly_base_face_set P2` `:> {fset 'poly_n}`.
≡ coercion to 'poly_n ≡ coercion to 'poly_n

A DEFINITION OF FACES

Over the type `{poly base}`, the definition of the faces is **straightforward**!

Definition `poly_base_face_set` ($P : \{\text{poly base}\}$) :=
[set $F : \{\text{poly base}\} \mid F \leq P$].

This definition is **independent** of the choice of base:

Lemma `face_set_morph` $\text{base1 base2} (P1 : \{\text{poly base1}\}) (P2 : \{\text{poly base2}\}) :$
 $P1 = P2 \Rightarrow \text{'poly_n} \rightarrow \text{poly_base_face_set } P1 = \text{poly_base_face_set } P2 \Rightarrow \{\text{fset 'poly_n}\}.$
 $\equiv$ coercion to 'poly_n $\equiv$ coercion to 'poly_n

We define

Definition `face_set` ($P : \text{'poly_n}$) :=
[fset ($F : \text{'poly_n}$) $\mid F \in \text{poly_base_face_set } (\backslash\text{repr } P)$].
 $\equiv$ a representative of P over an arbitrary
ambient H-representation

A DEFINITION OF FACES

Over the type `{poly base}`, the definition of the faces is **straightforward!**

Definition `poly_base_face_set` ($P : \{\text{poly base}\}$) :=
[set $F : \{\text{poly base}\} \mid F \leq P$].

This definition is **independent** of the choice of base:

Lemma `face_set_morph` $\text{base1 base2} (P1 : \{\text{poly base1}\}) (P2 : \{\text{poly base2}\}) :$
 $P1 = P2 \Rightarrow \text{poly_base_face_set } P1 = \text{poly_base_face_set } P2 \Rightarrow \{\text{fset 'poly_n}\}.$
 $\equiv$ coercion to 'poly_n $\equiv$ coercion to 'poly_n

We define

Definition `face_set` ($P : \text{'poly_n}$) :=
[fset ($F : \text{'poly_n}$) $\mid F \in \text{poly_base_face_set } (\backslash \text{repr } P)$].
 $\equiv$ a representative of P over an arbitrary
ambient H-representation

As it must be,

Theorem `face_setP` ($P F : \text{'poly_n}$) :=
 $\text{reflect } (F = [\text{poly0}] \wedge \exists c, F = \text{argmin } P c) (F \in \text{face_set } P).$

Idiomatic scheme. When dealing with the faces of $P : \text{'poly_n}$,

1. destruct P into a $\{\text{poly base}\}$ for an arbitrary ambient H-representation;
2. manipulate the faces F of P within the type $\{\text{poly base}\}$, and reduce the membership to $F \leq P$.

Idiomatic scheme. When dealing with the faces of $P : \text{'poly_n}$,

1. destruct P into a $\{\text{poly base}\}$ for an arbitrary ambient H-representation;
2. manipulate the faces F of P within the type $\{\text{poly base}\}$, and reduce the membership to $F \leq P$.

Lemma `face_set_polyI` ($P \ F1 \ F2 : \text{'poly_n}$) :

$F1 \ \text{\texttt{\textbackslash in}} \ \text{face_set } P \rightarrow F2 \ \text{\texttt{\textbackslash in}} \ \text{face_set } P \rightarrow F1 \ \text{\texttt{'\&'}} \ F2 \ \text{\texttt{\textbackslash in}} \ \text{face_set } P.$

Proof.

`elim/polybW`: $P \Rightarrow \text{base } P.$

`case/face_setP` $\Rightarrow \{\}F1 \ F1_sub_P.$

`case/face_setP` $\Rightarrow \{\}F2 \ F2_sub_P.$

`by rewrite` `face_setE` `?(le_trans leIl)` `?pvalP.`

`Qed.`

Idiomatic scheme. When dealing with the faces of $P : \text{'poly_n}$,

1. destruct P into a $\{\text{poly base}\}$ for an arbitrary ambient H-representation;
2. manipulate the faces F of P within the type $\{\text{poly base}\}$, and reduce the membership to $F \leq P$.

Lemma `face_set_polyI` ($P \ F1 \ F2 : \text{'poly_n}$) :

$F1 \ \text{\texttt{\textbackslash in}} \ \text{face_set } P \rightarrow F2 \ \text{\texttt{\textbackslash in}} \ \text{face_set } P \rightarrow F1 \ \text{\texttt{'\&'}} \ F2 \ \text{\texttt{\textbackslash in}} \ \text{face_set } P.$

Proof.

`elim/polybW`: $P \Rightarrow \text{base } P.$

`case/face_setP` $\Rightarrow \{\}F1 \ F1_sub_P.$

`case/face_setP` $\Rightarrow \{\}F2 \ F2_sub_P.$

`by rewrite` `face_setE` $?(le_trans \ leI1) \ ?pvalP.$

`Qed.`

introduce base and
destruct P into a $\{\text{poly base}\}$

EXAMPLE

Idiomatic scheme. When dealing with the faces of $P : \text{'poly_n}$,

1. destruct P into a $\{\text{poly base}\}$ for an arbitrary ambient H-representation;
2. manipulate the faces F of P within the type $\{\text{poly base}\}$, and reduce the membership to $F \leq P$.

Lemma `face_set_polyI` ($P \ F1 \ F2 : \text{'poly_n}$) :

$F1 \in \text{face_set } P \rightarrow F2 \in \text{face_set } P \rightarrow F1 \ \& \ F2 \in \text{face_set } P.$

Proof.

`elim/polybW`: $P \Rightarrow \text{base } P.$

`case/face_setP` $\Rightarrow \{\} F1 \ F1_sub_P.$

`case/face_setP` $\Rightarrow \{\} F2 \ F2_sub_P.$

`by rewrite` `face_setE` $?(le_trans \ leI1) \ ?pvalP.$

`Qed.`

lift $F1$ and $F2$ to $\{\text{poly base}\}$
and introduce $F1 \leq P$ and $F2 \leq P$

EXAMPLE

Idiomatic scheme. When dealing with the faces of $P : \text{'poly_n}$,

1. destruct P into a $\{\text{poly base}\}$ for an arbitrary ambient H-representation;
2. manipulate the faces F of P within the type $\{\text{poly base}\}$, and reduce the membership to $F \leq P$.

Lemma `face_set_polyI` ($P \ F1 \ F2 : \text{'poly_n}$) :

$F1 \in \text{face_set } P \rightarrow F2 \in \text{face_set } P \rightarrow F1 \ \& \ F2 \in \text{face_set } P.$

Proof.

`elim/polybW`: $P \Rightarrow \text{base } P.$

`case/face_setP` $\Rightarrow \{\}F1 \ F1_sub_P.$

`case/face_setP` $\Rightarrow \{\}F2 \ F2_sub_P.$

`by rewrite` `face_setE` `?(le_trans leI1)` `?pvalP.`

Qed.

rewrite the goal into $F1 \ \& \ F2 \leq P$
and prove it by transitivity

What's hidden.

$F1 \ \& \ F2 : \text{'poly_n}$ is automatically lifted to $\{\text{poly base}\}$ when $F1, F2 : \{\text{poly base}\}$:

- done using canonical structures, i.e., extra hints to the type inference system.

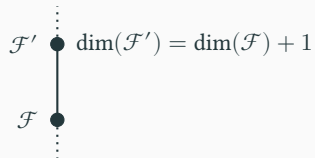
In this way, we show that `face_set P` has a structure of finite lattice

- the partial order is the polyhedron inclusion '`<=`'
- the meet operator is the intersection '`&`'
- the bottom and top elements are respectively `[poly0]` and `P`

In this way, we show that `face_set P` has a structure of finite lattice

Gradedness. We show that the face lattice is graded, using the dimension as a rank function.

⇒ relies on the formalization of affine spaces and hulls
(and their dimension)



In this way, we show that $\text{face_set } P$ has a structure of finite lattice

Gradedness. We show that the face lattice is graded, using the dimension as a rank function.

⇒ relies on the formalization of affine spaces and hulls
(and their dimension)

Theorem (Closedness under intervals).

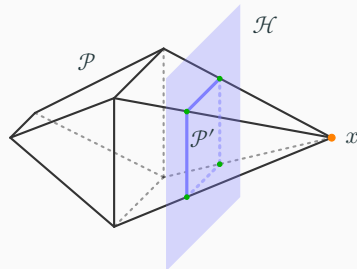
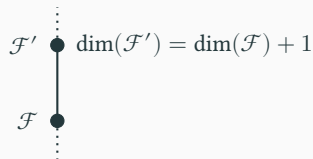
Let Q and Q' be two faces of a polytope P such that $Q \leq Q'$.

The sublattice formed by the faces

$$F \in \text{face_set } P \text{ satisfying } Q \leq F \leq Q'$$

is isomorphic to the face lattice of a polytope of dimension $\dim Q' - \dim Q$.

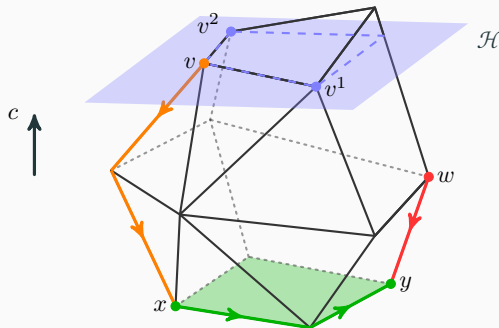
Vertex figure. $Q' = P$ and Q is a vertex x of P .



Theorem (Balinski, 1961). *The graph of a d -dimensional polytope is d -connected.*

Theorem balinski ($V : \{\text{fset } 'cV_n\}$) ($v\ w : 'cV_n$) :
 $\text{dim } P = n \rightarrow (V \text{ '}<=' \text{ vertex_set } P) \% \text{fset} \rightarrow \#|' V | = n.-1 \rightarrow$
 $v \setminus \text{in} (\text{vertex_set } P \setminus V) \% \text{fset} \rightarrow w \setminus \text{in} (\text{vertex_set } P \setminus V) \% \text{fset} \rightarrow$
 $\text{exists } p, [\wedge (\text{path } (\text{adj } P) \ v \ p), w = \text{last } v \ p \ \& \ \text{all } [\text{predC } V] \ p].$

The proof closely follows the one given in (Ziegler, 1995), exploiting a “geometric” version of the simplex method.



Computing with polyhedra

Fully computational approach.

We **implement** and **prove correct** an algorithm that “computes” the vertex-edge graph of a polytope (and its diameter) within Rocq.

Fully computational approach.

We **implement** and **prove correct** an algorithm that “computes” the vertex-edge graph of a polytope (and its diameter) within Rocq.

- **certificate-based**: the algorithm takes as input a graph computed by an informal software, and certifies that it is correct;

Fully computational approach.

We **implement** and **prove correct** an algorithm that “computes” the vertex-edge graph of a polytope (and its diameter) within Rocq.

- **certificate-based**: the algorithm takes as input a graph computed by an informal software, and certifies that it is correct;
- **simple** enough to make the correctness proof manageable;

Fully computational approach.

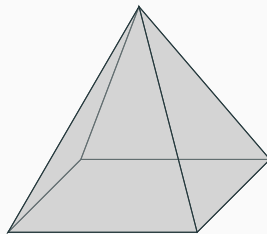
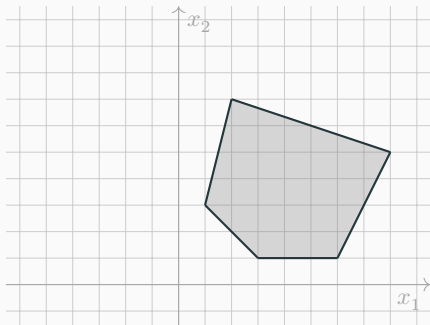
We **implement** and **prove correct** an algorithm that “computes” the vertex-edge graph of a polytope (and its diameter) within RocQ.

- **certificate-based**: the algorithm takes as input a graph computed by an informal software, and certifies that it is correct;
- **simple** enough to make the correctness proof manageable;
- **efficient** enough to deal with real-life polytopes, e.g., the non-Hirsch polytope of (Matschke et al., 2015) of dim. 20 and with 36 425 vertices, 364 250 edges and involving large rationals.

VERTICES VS FEASIBLE BASES

Reminder.

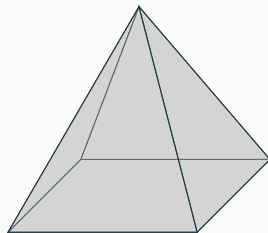
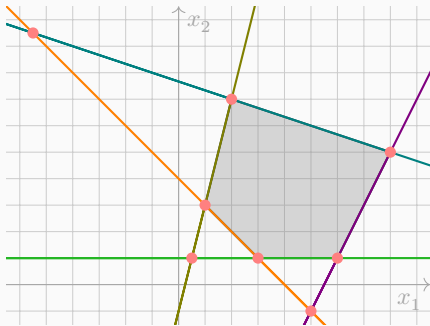
- A **basis** is a set of n supporting hyperplanes whose intersection is a single point (called a **basic point**).



VERTICES VS FEASIBLE BASES

Reminder.

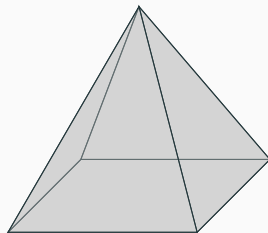
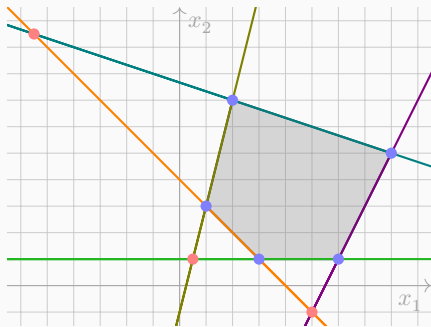
- A basis is a set of n supporting hyperplanes whose intersection is a single point (called a **basic point**).



VERTICES VS FEASIBLE BASES

Reminder.

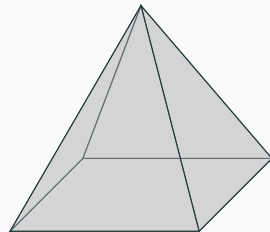
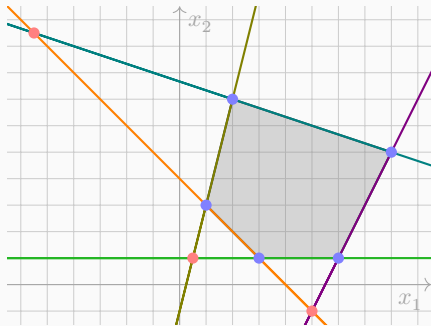
- A basis is a set of n supporting hyperplanes whose intersection is a single point (called a **basic point**).
- If the basic point belongs to the polyhedron, the basis is **feasible**.



VERTICES VS FEASIBLE BASES

Reminder.

- A basis is a set of n supporting hyperplanes whose intersection is a single point (called a **basic point**).
- If the basic point belongs to the polyhedron, the basis is **feasible**.

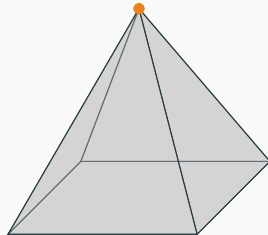
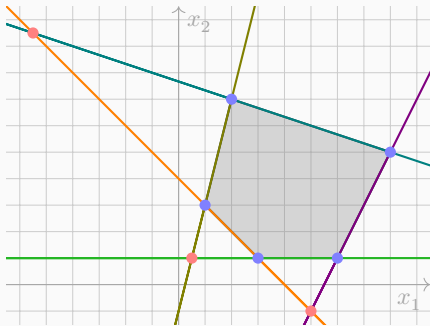


Proposition. Every vertex is a feasible basic point, i.e., a basic point associated with a feasible basis.

VERTICES VS FEASIBLE BASES

Reminder.

- A basis is a set of n supporting hyperplanes whose intersection is a single point (called a **basic point**).
- If the basic point belongs to the polyhedron, the basis is **feasible**.



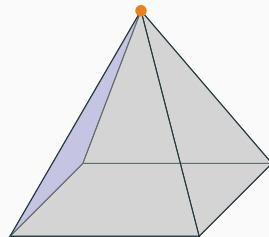
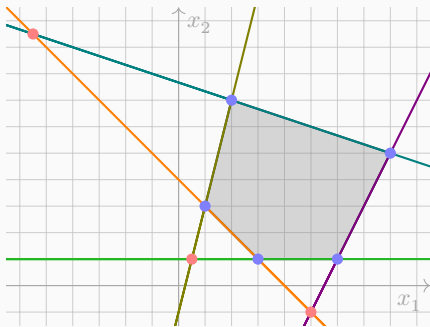
Proposition. Every vertex is a feasible basic point, i.e., a basic point associated with a feasible basis.

...but several bases may correspond to the same vertex.

VERTICES VS FEASIBLE BASES

Reminder.

- A basis is a set of n supporting hyperplanes whose intersection is a single point (called a **basic point**).
- If the basic point belongs to the polyhedron, the basis is **feasible**.



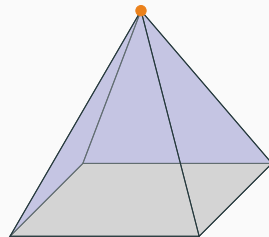
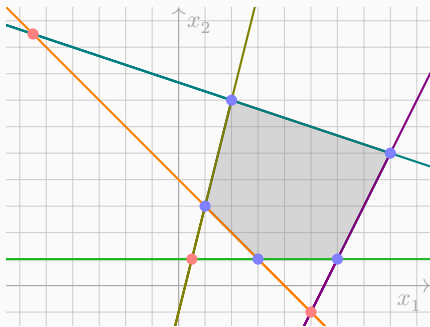
Proposition. *Every vertex is a feasible basic point, i.e., a basic point associated with a feasible basis.*

...but several bases may correspond to the same vertex.

VERTICES VS FEASIBLE BASES

Reminder.

- A basis is a set of n supporting hyperplanes whose intersection is a single point (called a **basic point**).
- If the basic point belongs to the polyhedron, the basis is **feasible**.



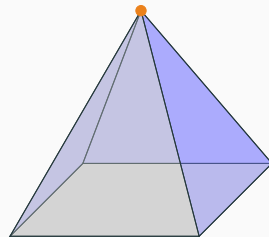
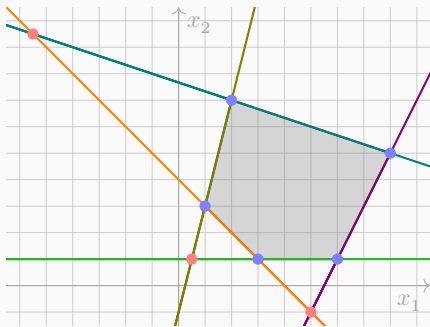
Proposition. Every vertex is a feasible basic point, i.e., a basic point associated with a feasible basis.

...but several bases may correspond to the same vertex.

VERTICES VS FEASIBLE BASES

Reminder.

- A basis is a set of n supporting hyperplanes whose intersection is a single point (called a **basic point**).
- If the basic point belongs to the polyhedron, the basis is **feasible**.



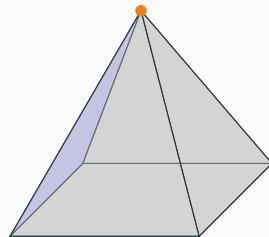
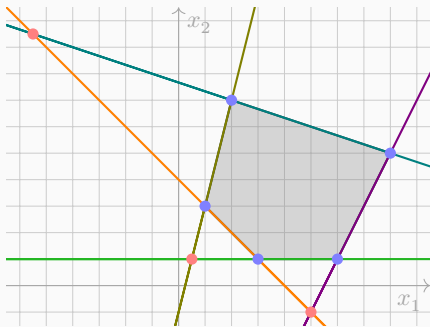
Proposition. Every vertex is a feasible basic point, i.e., a basic point associated with a feasible basis.

...but several bases may correspond to the same vertex.

VERTICES VS FEASIBLE BASES

Reminder.

- A basis is a set of n supporting hyperplanes whose intersection is a single point (called a **basic point**).
- If the basic point belongs to the polyhedron, the basis is **feasible**.



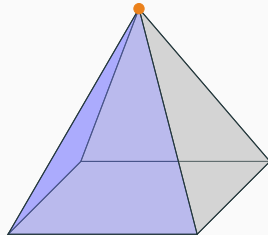
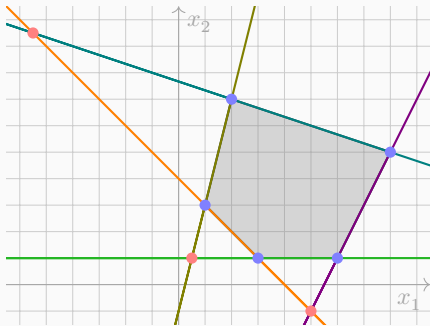
Proposition. Every vertex is a feasible basic point, i.e., a basic point associated with a feasible basis.

...but several bases may correspond to the same vertex.

VERTICES VS FEASIBLE BASES

Reminder.

- A basis is a set of n supporting hyperplanes whose intersection is a single point (called a **basic point**).
- If the basic point belongs to the polyhedron, the basis is **feasible**.



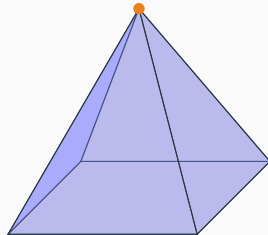
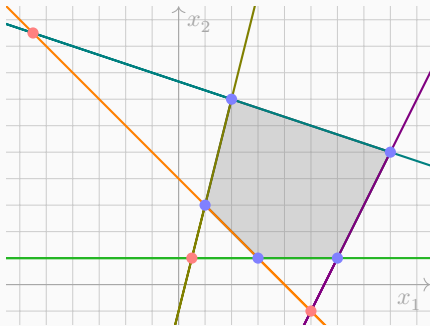
Proposition. Every vertex is a feasible basic point, i.e., a basic point associated with a feasible basis.

...but several bases may correspond to the same vertex.

VERTICES VS FEASIBLE BASES

Reminder.

- A basis is a set of n supporting hyperplanes whose intersection is a single point (called a **basic point**).
- If the basic point belongs to the polyhedron, the basis is **feasible**.



Proposition. *Every vertex is a feasible basic point, i.e., a basic point associated with a feasible basis.*

...but several bases may correspond to the same vertex.

Definition. We say that we are in the nondegenerate setting when there is a one-to-one correspondence between vertices and feasible bases.

Example. Simple polytopes described by facet-defining inequalities.

Definition. We say that we are in the nondegenerate setting when there is a one-to-one correspondance between vertices and feasible bases.

Example. Simple polytopes described by facet-defining inequalities.

Two feasible bases I and J are **adjacent** if they differ by **one** element:

$$\#(I \cap J) = n - 1.$$

VERTEX-EDGE GRAPH VS GRAPH OF FEASIBLE BASES

Definition. We say that we are in the nondegenerate setting when there is a one-to-one correspondence between vertices and feasible bases.

Example. Simple polytopes described by facet-defining inequalities.

Two feasible bases I and J are **adjacent** if they differ by **one** element:

$$\#(I \cap J) = n - 1.$$

Proposition. *In the nondegenerate setting,*

- *the vertex-edge graph G_{vert}*
- *the graph G_{bases} of feasible bases*

are isomorphic.

⇒ we first focus on certifying the graph of feasible bases, in the nondegenerate setting.

A FIRST ALGORITHM FOR THE NONDEGENERATE SETTING

Input. $G = (V, E)$ is an informally computed “graph of feasible bases”

```
1: assert  $G$  is nonempty
2: for all  $I \in V$  do assert  $I$  is a feasible basis
3: for all  $I \in V$  do
4:   for all  $J \in \text{neighborhood}_G(I)$  do assert  $\#(I \cap J) = n - 1$ 
5:   assert  $\#\text{neighborhood}_G(I) = n$ 
6: done
7: return true
```

A FIRST ALGORITHM FOR THE NONDEGENERATE SETTING

Input. $G = (V, E)$ is an informally computed “graph of feasible bases”

```
1: assert  $G$  is nonempty
2: for all  $I \in V$  do assert  $I$  is a feasible basis
3: for all  $I \in V$  do
4:   for all  $J \in \text{neighborhood}_G(I)$  do assert  $\#(I \cap J) = n - 1$ 
5:   assert  $\#\text{neighborhood}_G(I) = n$ 
6: done
7: return true
```

Theorem. Suppose $\mathcal{P}$ is a polytope in the nondegenerate setting. If the algorithm returns *true*, then $G = G_{\text{bases}}$.

A FIRST ALGORITHM FOR THE NONDEGENERATE SETTING

Input. $G = (V, E)$ is an informally computed “graph of feasible bases”

```
1: assert  $G$  is nonempty
2: for all  $I \in V$  do assert  $I$  is a feasible basis
3: for all  $I \in V$  do
4:   for all  $J \in \text{neighborhood}_G(I)$  do assert  $\#(I \cap J) = n - 1$ 
5:   assert  $\#\text{neighborhood}_G(I) = n$ 
6: done
7: return true
```

Theorem. Suppose $\mathcal{P}$ is a polytope in the nondegenerate setting. If the algorithm returns true, then $G = G_{\text{bases}}$.

Proof.

- $G \subset G_{\text{bases}}$;

A FIRST ALGORITHM FOR THE NONDEGENERATE SETTING

Input. $G = (V, E)$ is an informally computed “graph of feasible bases”

```
1: assert  $G$  is nonempty
2: for all  $I \in V$  do assert  $I$  is a feasible basis
3: for all  $I \in V$  do
4:   for all  $J \in \text{neighborhood}_G(I)$  do assert  $\#(I \cap J) = n - 1$ 
5:   assert  $\#\text{neighborhood}_G(I) = n$ 
6: done
7: return true
```

Corollary of Simplex.

- G_{bases} is a n -regular graph;

Theorem. Suppose $\mathcal{P}$ is a polytope in the nondegenerate setting. If the algorithm returns true, then $G = G_{\text{bases}}$.

Proof.

- $G \subset G_{\text{bases}}$;
- for all $I \in V$, $\text{neighborhood}_G(I) = \text{neighborhood}_{G_{\text{bases}}}(I)$ (inclusion + cardinality equality).

A FIRST ALGORITHM FOR THE NONDEGENERATE SETTING

Input. $G = (V, E)$ is an informally computed “graph of feasible bases”

```
1: assert  $G$  is nonempty
2: for all  $I \in V$  do assert  $I$  is a feasible basis
3: for all  $I \in V$  do
4:   for all  $J \in \text{neighborhood}_G(I)$  do assert  $\#(I \cap J) = n - 1$ 
5:   assert  $\#\text{neighborhood}_G(I) = n$ 
6: done
7: return true
```

Corollary of Simplex.

- G_{bases} is a n -regular graph;
- G_{bases} is connected.

Theorem. Suppose $\mathcal{P}$ is a polytope in the nondegenerate setting. If the algorithm returns true, then $G = G_{\text{bases}}$.

Proof.

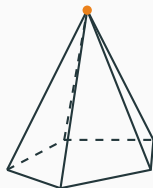
- $G \subset G_{\text{bases}}$;
- for all $I \in V$, $\text{neighborhood}_G(I) = \text{neighborhood}_{G_{\text{bases}}}(I)$ (inclusion + cardinality equality).

Fix $I_0 \in V$. Every feasible basis is connected to I_0 by a path in G_{bases} . This path is included in G .
 $\Rightarrow$ every feasible basis is in V .

THE GENERAL CASE VIA LEXICOGRAPHIC PERTURBATION

We use the lexicographic perturbation approach to reduce to the nondegenerate setting.

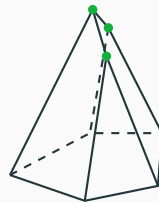
$$\begin{aligned} A_1 x &\geq b_1 \\ A_2 x &\geq b_2 \\ &\vdots \\ A_m x &\geq b_m \end{aligned}$$



$$0 < \varepsilon \ll 1$$

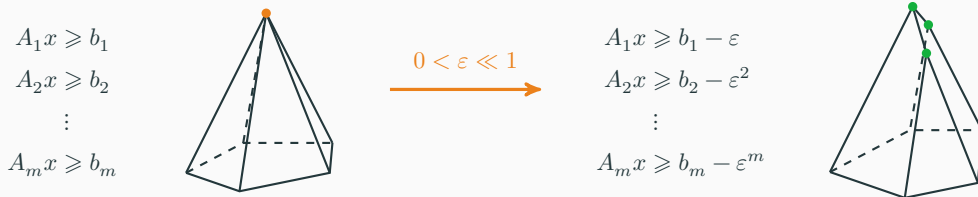


$$\begin{aligned} A_1 x &\geq b_1 - \varepsilon \\ A_2 x &\geq b_2 - \varepsilon^2 \\ &\vdots \\ A_m x &\geq b_m - \varepsilon^m \end{aligned}$$



THE GENERAL CASE VIA LEXICOGRAPHIC PERTURBATION

We use the lexicographic perturbation approach to reduce to the nondegenerate setting.



Symbolic approach.

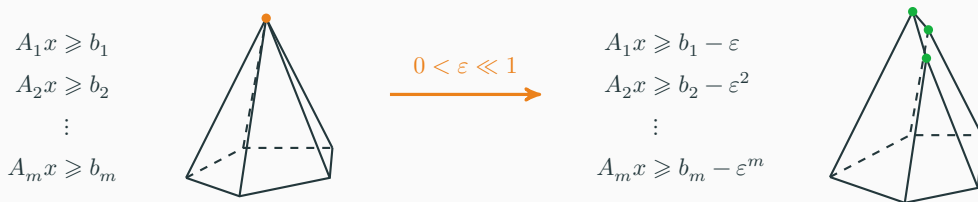
- moves the basic points in a very controlled way:

$$\begin{aligned} \tilde{x}^I &= A_I^{-1} \tilde{b}_I = A_I^{-1} [b_i - \varepsilon^i]_{i \in I} \\ &= x^I - \sum_{i \in I} \varepsilon^i \times (i\text{-th column of } A_I^{-1}) \end{aligned}$$

vector of polynomials in $\mathbb{R}_{\leq m}[\varepsilon]$

THE GENERAL CASE VIA LEXICOGRAPHIC PERTURBATION

We use the lexicographic perturbation approach to reduce to the nondegenerate setting.



Symbolic approach.

- moves the basic points in a very controlled way:

$$\begin{aligned} \tilde{x}^I &= A_I^{-1} \tilde{b}_I = A_I^{-1} [b_i - \varepsilon^i]_{i \in I} \\ &= x^I - \sum_{i \in I} \varepsilon^i \times (i\text{-th column of } A_I^{-1}) \end{aligned}$$

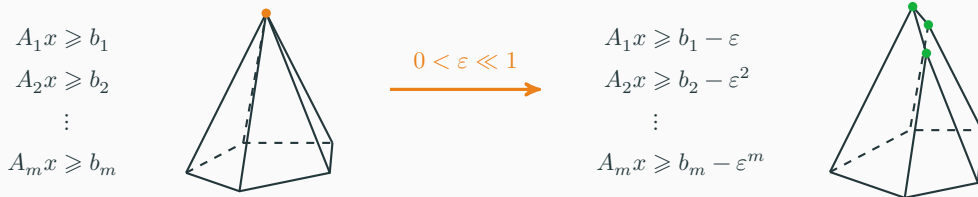
vector of polynomials in $\mathbb{R}_{\leq m}[\varepsilon]$

- turns feasibility into a set of **lexicographic** inequalities

$$\forall i \in [m], \quad A_i \tilde{x}^I \geq b_i - \varepsilon^i \iff \text{coeffs of } \varepsilon^0, \dots, \varepsilon^m \text{ in } A_i \tilde{x}^I \geq_{\text{lex}} (b_i, 0, \dots, 0, -1, 0, \dots, 0)$$

THE GENERAL CASE VIA LEXICOGRAPHIC PERTURBATION

We use the lexicographic perturbation approach to reduce to the nondegenerate setting.



Denote $\tilde{\mathcal{P}}$ the symbolically perturbed polyhedron.

Theorem. The graph of $\mathcal{P}$ is the image of the graph of $\tilde{\mathcal{P}}$ under the map

$$\phi: \tilde{x} \in (\mathbb{R}[\varepsilon])^n \mapsto \tilde{x}(0).$$

Input. An informal graph of pairs (lex-feasible basis I , perturbed basic point x).

Definition `graph_certification` $G : \text{bool} :=$
 $(G \neq \text{graph0}) \ \&\&$
 $[\text{forall } '(I,x) : \text{vertices } G,$
 $\quad [\&\& \#I == n,$
 $\quad \quad (\text{row_submx } A \ I) * m \ x == \text{row_submx } b_pert \ I,$
 $\quad \quad A * m \ x \geq_{\text{lex}} b_pert,$
 $\quad \quad [\text{forall } (J,_) : \text{successors } G \ (I,x), \ \#|I : \&: J| == n-1]$
 $\quad \quad \& \#|' \text{successors } G \ (I,x)| == n$
 $\quad]$
 $].$

Input. An informal graph of pairs (lex-feasible basis I , perturbed basic point x).

Definition `graph_certification` $G : \text{bool} :=$

$(G \neq \text{graph0}) \ \&\&$

`[forall` $'(I,x) : \text{vertices } G,$

$\&\& \#I == n,$

$(\text{row_submx } A \ I) * m \ x == \text{row_submx } b_pert \ I, \quad I \text{ is a lex-feasible basis}$
 $A * m \ x \geq_{\text{lex}} b_pert,$

`[forall` $(J,_) : \text{successors } G \ (I,x), \ \#|I : \&: J| == n-1]$

$\& \#|' \text{successors } G \ (I,x)| == n$

`]`

`].`

Input. An informal graph of pairs (lex-feasible basis I , perturbed basic point x).

Definition `graph_certification` $G : \text{bool} :=$

$(G \neq \text{graph0}) \ \&\&$

`[forall '(I,x) : vertices G,`

`[&& #I == n,`

`(row_submx A I) *m x == row_submx b_pert I,   I is a lex-feasible basis`
`A *m x >=lex b_pert,`

`[forall (J,_) : successors G (I,x), #|I :&: J| == n-1]   adjacency`

`& #|' successors G (I,x)| == n`

`]`

`].`

Input. An informal graph of pairs (lex-feasible basis I , perturbed basic point x).

Definition `graph_certification` $G : \text{bool} :=$

$(G \neq \text{graph0}) \ \&\&$

`[forall '(I,x) : vertices G,`

`[&& #I == n,`

`(row_submx A I) *m x == row_submx b_pert I,`

`A *m x >=lex b_pert,`

`[forall (J,_) : successors G (I,x), #|I :&: J| == n-1]`

`& #|' successors G (I,x)| == n`

`]`

`].`

I is a lex-feasible basis

adjacency

$G \subset \text{lex_graph}$

Input. An informal graph of pairs (lex-feasible basis I , perturbed basic point x).

Definition `graph_certification` $G : \text{bool} :=$

$(G \neq \text{graph0}) \ \&\&$

$[\text{forall } '(I,x) : \text{vertices } G,$

$[\&\& \#I == n,$

$(\text{row_submx } A \ I) * m \ x == \text{row_submx } b_pert \ I,$
 $A * m \ x \geq_{lex} b_pert,$

I is a lex-feasible basis

$[\text{forall } (J,_) : \text{successors } G \ (I,x), \ \#|I : \&: J| == n-1]$

adjacency

$\& \ \#| \text{successors } G \ (I,x)| == n$ neighborhood cardinality

$G \subset \text{lex_graph}$

$]$

$].$

Input. An informal graph of pairs (lex-feasible basis I , perturbed basic point x).

Definition `graph_certification` $G : \text{bool} :=$

`(G != graph0) &&`

`[forall '(I,x) : vertices G,`

`[&& #I == n,`

`(row_submx A I) *m x == row_submx b_pert I,`

`A *m x >=lex b_pert,`

I is a lex-feasible basis

`[forall (J,_) : successors G (I,x), #|I :&: J| == n-1]`

adjacency

`& #|' successors G (I,x)| == n` neighborhood cardinality

$G \subset \text{lex_graph}$

`]`
`].`

In this way,

Theorem `repr_lex_graph` $G :$

`graph_certification G = true -> giso G lex_graph.`

where `lex_graph` is the adjacency graph of lex-feasible bases.

Input. An informal graph of pairs (lex-feasible basis I , perturbed basic point x).

Definition `graph_certification` $G : \text{bool} :=$

$(G \neq \text{graph0}) \ \&\&$

`[forall '(I,x) : vertices G,`

`[&& #I == n,`

`(row_submx A I) *m x == row_submx b_pert I,`

`A *m x >=lex b_pert,`

I is a lex-feasible basis

`[forall (J,_) : successors G (I,x), #|I :&: J| == n-1]`

adjacency

`& #|' successors G (I,x)| == n` neighborhood cardinality

$G \subset \text{lex_graph}$

`]`
`].`

In this way,

Theorem `repr_lex_graph` $G :$

`graph_certification G = true -> giso G lex_graph.`

where `lex_graph` is the adjacency graph of lex-feasible bases.

Theorem `repr_poly_graph` $G :$

`graph_certification G = true -> graph P = (phi @/ G).`

Data types used for proofs are **not made for computations**:

- Peano (unary) naturals
- list-based sets, graphs, vectors, matrices
- inductive types carrying data **and** invariants

DATATYPES: PROOF VS COMPUTATION

Data types used for proofs are **not made for computations**:

- Peano (unary) naturals
- list-based sets, graphs, vectors, matrices
- inductive types carrying data **and** invariants

We **refine** the algorithm to work over computation oriented types:

(i) machine integers; (ii) persistent arrays; (iii) bignum arithmetic (Grégoire and Théry, 2006)

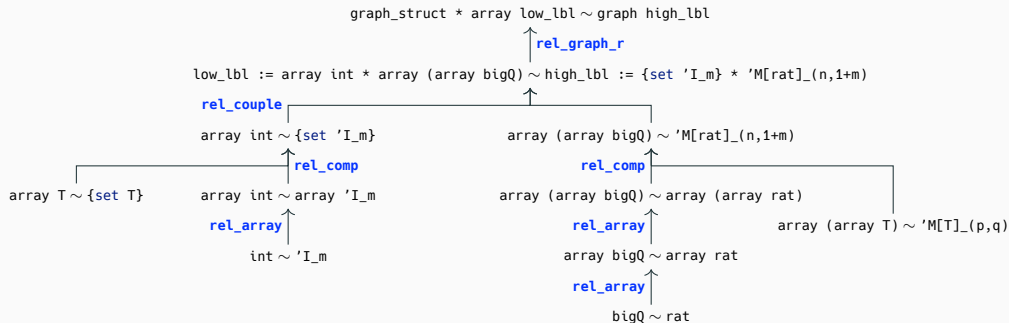
DATATYPES: PROOF VS COMPUTATION

Data types used for proofs are **not made for computations**:

- Peano (unary) naturals
- list-based sets, graphs, vectors, matrices
- inductive types carrying data **and** invariants

We **refine** the algorithm to work over computation oriented types:

(i) machine integers; (ii) persistent arrays; (iii) bignum arithmetic (Grégoire and Théry, 2006)



Input of the checker. A certificate generated from the output of `lrs` (Avis) containing:

- a n -dim. polytope described by m inequalities
- the adjacency graph of its lexicographic bases
- the lex-feasible basic points; each is $n \times (1 + m)$ -matrix of big rationals
- the vertex-edge graph

Input of the checker. A certificate generated from the output of `lrs` (Avis) containing:

- a n -dim. polytope described by m inequalities
- the adjacency graph of its lexicographic bases
- the lex-feasible basic points; each is $n \times (1 + m)$ -matrix of big rationals
- the vertex-edge graph

Counterexamples to the Hirsch conjecture (Matschke et al., 2015).

instance	dimension	# ineq.	# vertices	# edges
<code>poly20dim21</code>	20	40	36 425	364 250
<code>poly23dim24</code>	23	46	73 224	842 076

Compiling certificates from plain Rocq files?

- `poly23dim24`: file size exceeds 600 MB, run out of memory on a 64 MB machine.

Input of the checker. A certificate generated from the output of `lrs` (Avis) containing:

- a n -dim. polytope described by m inequalities
- the adjacency graph of its lexicographic bases
- the lex-feasible basic points; each is $n \times (1 + m)$ -matrix of big rationals
- the vertex-edge graph

Counterexamples to the Hirsch conjecture (Matschke et al., 2015).

instance	dimension	# ineq.	# vertices	# edges
poly20dim21	20	40	36 425	364 250
poly23dim24	23	46	73 224	842 076

Compiling certificates from plain Rocq files?

- `poly23dim24`: file size exceeds 600 MB, run out of memory on a 64 MB machine.

⇒ implementation of a plugin `binreader` that constructs a term from a binary file.

- `poly23dim24`: 19 min, max. memory usage 50 GB.

EXPERIMENTS (2)

```
Theorem Hirsch_was_wrong : (* Santos was right :-) *)  
  exists (d : nat) (P : 'polytope[rat]_d),  
    diameter (graph P) > #|'facets P| - (dim P).
```

Proof.

```
pose P := poly_of_syst (A, b).
```

```
exists d'.+1, P.
```

```
apply/disprove_Hirsch.
```

- exact: well_formedness_test.
- exact: lex_graph_test.
- exact: img_graph_test.
- exact: boundedness_test.
- exact: full_dim_test.
- exact: diameter_test.

Qed.

EXPERIMENTS (2)

```
Theorem Hirsch_was_wrong : (* Santos was right :-) *)  
  exists (d : nat) (P : 'polytope[rat]_d),  
    diameter (graph P) > #|'facets P| - (dim P).
```

Proof.

```
pose P := poly_of_syst (A, b).
```

```
exists d'.+1, P.
```

```
apply/disprove_Hirsch.
```

```
- exact: well_formedness_test.
```

```
- exact: lex_graph_test.
```

```
- exact: img_graph_test.
```

```
- exact: boundedness_test.
```

```
- exact: full_dim_test.
```

```
- exact: diameter_test.
```

Qed.

New “theorems”:

```
Theorem poly20dim21_diameter :
```

```
  diameter (poly_graph poly20dim21) = 21%nat.
```

```
Theorem poly23dim24_diameter :
```

```
  diameter (poly_graph poly23dim24) = 24%nat.
```

Computations are carried out by the Rocq reduction strategy `vm_compute`:

- Rocq code is translated into bytecode instructions, interpreted by a VM;
- 20x – 50x slower than (extracted) OCaml compiled code.

Computations are carried out by the Rocq reduction strategy `vm_compute`:

- Rocq code is translated into bytecode instructions, interpreted by a VM;
- 20x – 50x slower than (extracted) OCaml compiled code.

instance	lrs (s)	certificate size (MB)	certificate loading (vs lrs)	certificate checking (vs lrs)
cube_14	0.29	257	303.69	600.45
cube_15	0.58	582	319.09	691.55
cube_16	1.26	1308	319.71	758.64
cross_7	0.29	348	473.90	1192.83
cross_8	1.87	6181	1597.32	4308.60
dual_cyclic_11_22	0.53	98	101.64	309.83
dual_cyclic_12_24	1.78	322	89.19	339.33
dual_cyclic_13_26	4.86	817	94.34	402.72
poly20dim21	8.00	1228	53.50	569.36
poly23dim24	13.29	2977	85.53	439.22

Computations are carried out by the Rocq reduction strategy `vm_compute`:

- Rocq code is translated into bytecode instructions, interpreted by a VM;
- 20x – 50x slower than (extracted) OCaml compiled code.

instance	lrs (s)	certificate size (MB)	certificate loading (vs lrs)	certificate checking (vs lrs)
cube_14	0.29	257	303.69	600.45
cube_15	0.58	582	319.09	691.55
cube_16	1.26	1308	319.71	758.64
cross_7	0.29	348	473.90	1192.83
cross_8	1.87	6181	1597.32	4308.60
dual_cyclic_11_22	0.53	98	101.64	309.83
dual_cyclic_12_24	1.78	322	89.19	339.33
dual_cyclic_13_26	4.86	817	94.34	402.72
poly20dim21	8.00	1228	53.50	569.36
poly23dim24	13.29	2977	85.53	439.22

Bottlenecks. In decreasing order:

1. **numerical part** of the certificate checking: for each perturbed basic point $\tilde{x}$ (1 per lex-feasible basis),

lex-compare $A\tilde{x}$ with $\tilde{b}$ $\implies O(m^2n)$ operations over rational numbers

Computations are carried out by the Rocq reduction strategy `vm_compute`:

- Rocq code is translated into bytecode instructions, interpreted by a VM;
- 20x – 50x slower than (extracted) OCaml compiled code.

instance	lrs (s)	certificate size (MB)	certificate loading (vs lrs)	certificate checking (vs lrs)
cube_14	0.29	257	303.69	600.45
cube_15	0.58	582	319.09	691.55
cube_16	1.26	1308	319.71	758.64
cross_7	0.29	348	473.90	1192.83
cross_8	1.87	6181	1597.32	4308.60
dual_cyclic_11_22	0.53	98	101.64	309.83
dual_cyclic_12_24	1.78	322	89.19	339.33
dual_cyclic_13_26	4.86	817	94.34	402.72
poly20dim21	8.00	1228	53.50	569.36
poly23dim24	13.29	2977	85.53	439.22

Bottlenecks. In decreasing order:

1. numerical part of the certificate checking
2. loading the certificate

Summary. 500x – 5,000x slower than informal enumeration of vertices (lex-feasible bases) by lrs.

Concluding remarks

Summary.

- lots of fun for people at the crossroads of maths and software
- finding the “right” definitions, and make them play together nicely
- computation in proof assistants make **certification problems** even more important

Summary.

- lots of fun for people at the crossroads of maths and software
- finding the “right” definitions, and make them play together nicely
- computation in proof assistants make **certification problems** even more important

Long-term goal. Formal certification should not take (much) longer than informal computation!

Summary.

- lots of fun for people at the crossroads of maths and software
- finding the “right” definitions, and make them play together nicely
- computation in proof assistants make **certification problems** even more important

Long-term goal. Formal certification should not take (much) longer than informal computation!

Work in progress (joint w/ Yazid Id-Sahra and Pierre-Yves Strub).

A new approach to certify the vertices / graph of polyhedra:

- reduce the numerical burden to membership tests of the candidate vertices
- rely on a mod-2 homology-based criterion to check completeness
- lead to small certificates
- computation with `vm_compute` is about 5x-10x slower than informal computation
- extracted OCaml code is **faster** than informal computation

THANK YOU!

A formalization of convex polyhedra based on the simplex method

arXiv:1706.10269

Formalizing the Face Lattice of Polyhedra

arXiv:2104.15021

A Formal Disproof of the Hirsch Conjecture

arXiv:2301.04060

REFERENCES

- D. Avis. The lrslib Library. <http://cgm.cs.mcgill.ca/~avis/C/lrs.html>.
- Michel L. Balinski. On the graph structure of convex polyhedra in n -space. Pacific Journal of Mathematics, 11(2):431 – 434, 1961. doi: [pjm/1103037323](https://doi.org/10.2307/23723). URL <https://doi.org/>.
- Cyril Cohen. Pragmatic quotient types in Coq. In Sandrine Blazy, Christine Paulin-Mohring, and David Pichardie, editors, Interactive Theorem Proving, pages 213–228, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg. ISBN 978-3-642-39634-2.
- George B. Dantzig, Alex Orden, and Philip Wolfe. The generalized simplex method for minimizing a linear form under linear inequality restraints. Pacific J. Math., 5(2), 1955.
- Georges Gonthier, Assia Mahboubi, and Enrico Tassi. A Small Scale Reflection Extension for the Coq system. Research Report RR-6455, Inria Saclay Ile de France, 2016.
- Benjamin Grégoire and Laurent Théry. A purely functional library for modular arithmetic and its application to certifying large prime numbers. In Automated Reasoning, Third International Joint Conference, IJCAR 2006, Seattle, WA, USA, August 17-20, 2006, Proceedings, volume 4130 of Lecture Notes in Computer Science, pages 423–437. Springer, 2006.
- Benjamin Matschke, Francisco Santos, and Christophe Weibel. The width of five-dimensional prisms. Proceedings of the London Mathematical Society, 110(3):647–672, 2015. ISSN 1460-244X. doi: [10.1112/plms/pdu064](https://doi.org/10.1112/plms/pdu064). URL <http://dx.doi.org/10.1112/plms/pdu064>.
- Francisco Santos. A counterexample to the Hirsch conjecture. Ann. Math., 176(1), 2012. ISSN 0003-486X; 1939-8980/e.

Günter M. Ziegler. Lectures on polytopes. Springer-Verlag, 1995.